

Title: Phishing Website Detection Using Machine Learning

Done by : Akshada Borhade

Project Number : 04, Submission Date: 13th June 2025

Overview:

Phishing attacks are one of the most common and damaging cybersecurity threats, where attackers trick users into providing sensitive information by mimicking legitimate websites. The goal of this project is to develop a machine learning model that can automatically detect phishing websites based on various website features. By analyzing characteristics like URL structure, domain registration, and web traffic, the model classifies websites as either phishing or legitimate. This automated detection helps protect users and organizations from fraud and cyberattacks by blocking malicious sites before harm occurs.

Libraries used:

- pandas: For loading, manipulating, and preprocessing tabular data.
- numpy: For numerical operations and array handling.
- scikit-learn: Includes tools for preprocessing, model training, evaluation, and data splitting.
- xgboost: Implements the powerful gradient boosting tree algorithm used for classification.
- matplotlib / seaborn: For visualizing data and model performance (optional but helpful).

Dataset Details:

The dataset used for this project contains numerous features extracted from websites, such as URL length, presence of IP addresses, domain registration length, SSL state, web traffic rank, and other metadata relevant to phishing detection. Each record is labeled as either phishing (-1) or legitimate (1). The dataset contains thousands of samples collected from various sources, ensuring diverse representation of phishing tactics and legitimate websites. I have taken the dataset from Kaggle for which the link is

https://www.kaggle.com/datasets/omkargowda/phishing-website-dataset?select=Phising_Training_Dataset.csv

APIs Integrated:

Not Applicable

Source code 1:

File Name : Setup, Load and Explore the Dataset

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from xgboost import XGBClassifier
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Load datasets
train_df = pd.read_csv('Phising_Training_Dataset.csv')
test_df = pd.read_csv('Phising_Testing_Dataset.csv')
```

Source code 2: Data Preprocessing and Cleaning

```
# Drop 'key' or index column if present
train_df = train_df.drop(columns=['key'], errors='ignore')
test_df = test_df.drop(columns=['key'], errors='ignore')

# Separate features and target variable
X = train_df.drop('Result', axis=1)
y = train_df['Result']

# Convert labels from {-1, 1} to {0, 1} for model compatibility
y = y.replace(-1, 0)

# Repeat for test target if available
if 'Result' in test_df.columns:
    y_test = test_df['Result'].replace(-1, 0)
    X_test = test_df.drop('Result', axis=1)
else:
    X_test = test_df
```

Source code 3: Train-Test Split and Feature Scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
# Split train data into train and validation sets
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

Source code 4: Model Training Using XGBoost

```
# --- MODEL TRAINING AND EVALUATION ---

def evaluate_model(model, X_val, y_val, title="Model"):
    y_pred = model.predict(X_val)

    print(f"--- {title} Classification Report ---")

    print(classification_report(y_val, y_pred))

    cm = confusion_matrix(y_val, y_pred)

    plt.figure(figsize=(5,4))

    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

    plt.title(f"{title} Confusion Matrix")

    plt.xlabel('Predicted')

    plt.ylabel('Actual')

    plt.show()

# 3. XGBoost

xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss',
    random_state=42)

xgb.fit(X_train_scaled, y_train)

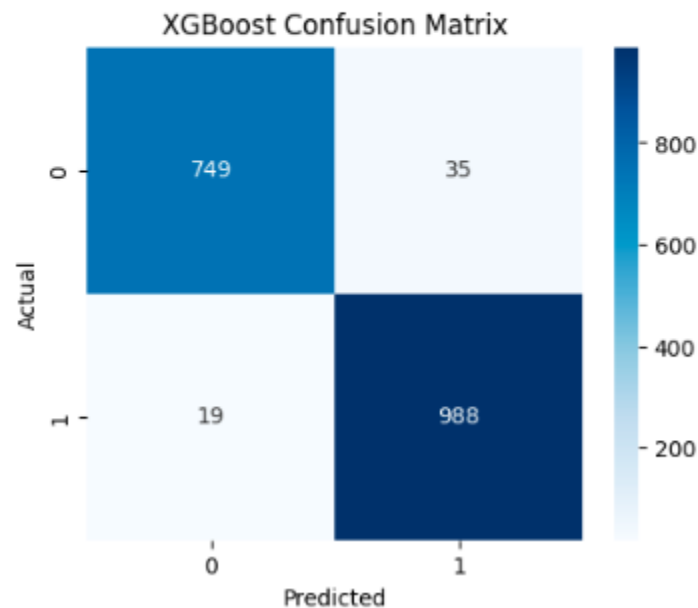
evaluate_model(xgb, X_val_scaled, y_val, "XGBoost")
```

Output Screenshots:

```
--- XGBoost Classification Report ---
              precision    recall  f1-score   support

     0       0.98        0.96        0.97        784
     1       0.97        0.98        0.97       1007

 accuracy          0.97          0.97          0.97       1791
 macro avg         0.97          0.97          0.97       1791
 weighted avg      0.97          0.97          0.97       1791
```



What I learned:

- How to handle and preprocess real-world cybersecurity data for machine learning.
- The importance of feature scaling and label encoding for tree-based models.
- How gradient boosting algorithms like XGBoost can effectively classify phishing websites.
- The critical role of splitting datasets properly to evaluate model generalization.
- Techniques for evaluating classification models using accuracy, precision, recall, and F1-score.
- Understanding of phishing characteristics and their representation as machine-learnable features.

What Skills I gained:

- Data cleaning and preprocessing with pandas and sklearn.

- Label transformation and handling imbalanced binary classification.
- Feature scaling and train-validation splitting for robust model evaluation.
- Implementing and tuning tree-based ensemble methods like XGBoost.
- Using classification metrics and confusion matrices for performance assessment

Further Enhancement Suggestions:

- Experiment with other ensemble models like Random Forest, LightGBM, or CatBoost.
- Use feature selection or dimensionality reduction to improve model speed and accuracy.
- Apply deep learning models on website content (HTML/text) for hybrid phishing detection.
- Implement real-time phishing detection integrated with browser plugins or fire-wall.
- Tune hyperparameters with grid search or Bayesian optimization for best results.

Real Time Applications:

- Proactively blocking phishing websites in browsers or network filters.
- Protecting users from fraud and identity theft by identifying suspicious sites.
- Helping cybersecurity teams prioritize threats by automating phishing detection.
- Integrating with email filters to detect phishing links in messages.
- Enhancing cybersecurity awareness by analyzing trends in phishing attack methods.